



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Formal Verification and Validation

Toby Murray



toby.murray@unimelb.edu.au



DMD 8.17 (Level 8, Doug McDonnell Bldg)

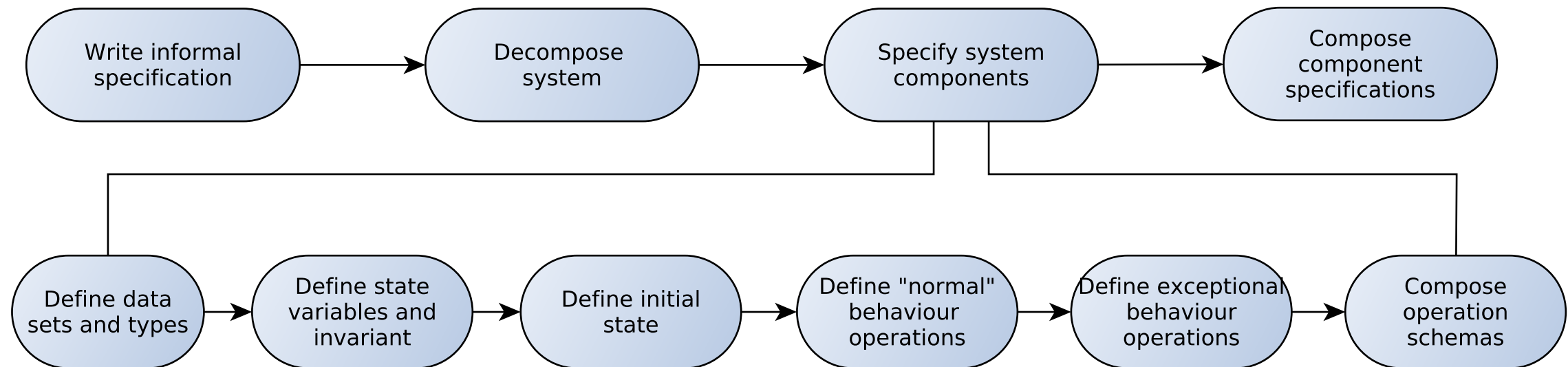


<http://people.eng.unimelb.edu.au/tobym>



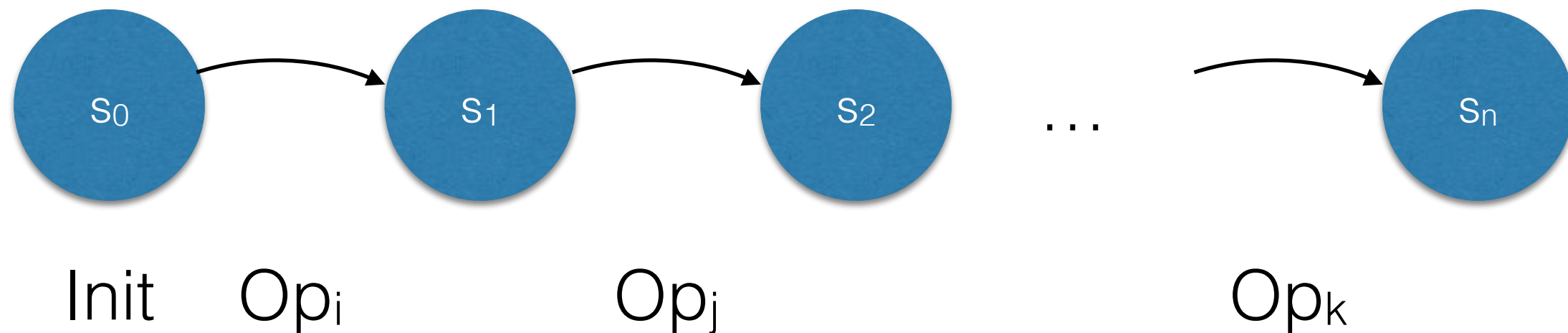
@tobycmurray

Specification Process



State Invariants

How to check a predicate $Inv[s]$ of all states s in a state machine?



assert $invInit$ { **all** $s : State \mid Init[s] \Rightarrow Inv[s]$ }
assert $invOp_i$ { **always all** $s : State \mid Inv[s] \textbf{ and } Op_i[s] \Rightarrow$
 after $Inv[s]$ }
(for each operation op_i)

Exceptional Behaviour



Distinguishing normal and exceptional behaviour in models

```
//Add a password for a *new* user/url pair
pred addNormal [pb : PassBook, url : URL, user: Username,
                pwd: Password, report : one Report] {
    addPre [pb, url, user]
    addPost [pb, url, user, pwd]
    report in Success
}

//Fail to add a password that already exists
pred addExceptional [pb : PassBook, url : URL, user: Username,
                    report : one Report] {
    not addPre [pb, url, user]
    pb.password' = pb.password
    report in Failed
}

//Add a password for a *new* user/url, otherwise, add nothing
pred add [pb : PassBook, url : URL, user: Username, pwd: Password,
          report : one Report] {
    addNormal [pb, url, user, pwd, report]
    or
    addExceptional [pb, url, user, report]
}
```

Exceptional Behaviour

The point is to constrain what the system
is meant to do in exceptional cases

```
//Fail to add a password that already exists
pred addExceptional [pb : PassBook, url : URL, user: Username,
                    report : one Report] {
    not addPre [pb, url, user]
    pb.password' = pb.password
    report in Failed
}
```

Exceptional Behaviour

Model it **separately** from normal behaviour

Where does it come from?

e.g. Hazard Analysis (HAZOP) etc.

Framing

How are these two predicates different?

```
//Fail to add a password that already exists
pred addExceptional [pb : PassBook, url : URL, user: Username,
                    report : one Report] {
    not addPre [pb, url, user]
    pb.password' = pb.password
    report in Failed
}
```

```
//Fail to add a password that already exists
pred addExceptional [pb : PassBook, url : URL, user: Username,
                    report : one Report] {
    not addPre [pb, url, user]
    //pb.password' = pb.password
    report in Failed
}
```

Second allows pb.password in the post-state to be **anything**,
and so allows add's exceptional behaviour to be arbitrary

What does this spec say?

Imagine that **Obj**s have two variable fields, **x** and **y**

```
pred addToX [o : Obj] {  
  o.x' = o.x + ...  
}
```

Allows the **y** field to change arbitrarily. It should be:

```
pred addToX [o : Obj] {  
  o.x' = o.x + ...  
  o.y' = o.y  
}
```




V&V FOR SPECS

V&V for formal specs

How do we do V&V for our specs?

Animation (e.g. Alloy's **run** command)

Model Checking (e.g. Alloy's **check** command)

Implement and test

Reviews and Inspections

Proof

Proof: An Example

$\text{add}[\text{pb}, \text{user}, \text{url}, \text{pwd}, \text{res}] =$
 $\text{addNormal}[\text{pb}, \text{user}, \text{url}, \text{pwd}, \text{res}]$ **or**
 $\text{addExceptional}[\text{pb}, \text{user}, \text{url}, \text{res}]$

all $\text{pb}, \text{user}, \text{url}, \text{pwd}, \text{res} \mid$
 $\text{Inv}[\text{pb}]$ **and** $\text{add}[\text{pb}, \text{user}, \text{url}, \text{pwd}, \text{res}] \Rightarrow$
after $\text{Inv}[\text{pb}]$

Proof: An Example

all pb, user, url, pwd, res |
 Inv[pb] **and**
 (addNormal[pb, user, url, pwd, res] **or**
 addExceptional[pb, user, url, res]) =>
after Inv[pb]

Proof: An Example

Consider both cases of the **or**

all pb, user, url, pwd, res |
 (Inv[pb] **and** addNormal[pb, user, url, pwd, res])
or
 (Inv[pb] **and** addExceptional[pb, user, url, res]) $\Rightarrow$
after Inv[pb]

Proof: An Example

Case 1

addNormal[pb, user, url pwd, res] =
 no pb.password[user][url]
 pb.password' = pb.password + (user->url->pwd)
 res in Success

all pb, user, url, pwd, res |
 Inv[pb] **and** addNormal[pb, user, url, pwd, res]) $\Rightarrow$
after Inv[pb]

Proof: An Example

Case 1

addNormal[pb, user, url pwd, res] =
 no pb.password[user][url]
 pb.password' = pb.password + (user->url->pwd)
 res in Success

all pb, user, url, pwd, res |
 Inv[pb] **and no** pb.password[user][url] **and**
 pb.password' = pb.password + (user->url->pwd) **=>**
after Inv[pb]

Proof: An Example

Case 1

$\text{Inv}[\text{pb}] = \mathbf{all} \text{ user, url} \mid \mathbf{lone} \text{ pb.password}[\text{user}][\text{url}]$

$\mathbf{all} \text{ pb, user, url, pwd, res} \mid$
 $\text{Inv}[\text{pb}] \mathbf{and no} \text{ pb.password}[\text{user}][\text{url}] \mathbf{and}$
 $\text{pb.password}' = \text{pb.password} + (\text{user} \rightarrow \text{url} \rightarrow \text{pwd}) \Rightarrow$
 $\mathbf{after} \text{ Inv}[\text{pb}]$

Proof: An Example

Case 1

Two cases to consider: $user_2, url_2$ are *user, url* or not

all *pb, user, url, pwd, res* |
 (**all** $user_1, url_1$ | **lone** $pb.password[user_1][url_1]$) **and**
 no $pb.password[user][url]$ **and**
 $pb.password' = pb.password + (user \rightarrow url \rightarrow pwd) \Rightarrow$
 (**all** $user_2, url_2$ | **lone** $pb.password'[user_2][url_2]$)

Proof: An Example

Case 1.1

$user_2, url_2$ are $user, url$

all $pb, user, url, pwd, res$ |
(**all** $user_1, url_1$ | **lone** $pb.password[user_1][url_1]$) **and**
no $pb.password[user][url]$ **and**
 $pb.password' = pb.password + (user \rightarrow url \rightarrow pwd) \Rightarrow$
(**lone** $pb.password'[user][url]$)

Proof: An Example

Case 1.2

$user_2, url_2$ are not $user, url$

all $pb, user, url, pwd, res$ |
 (**all** $user_1, url_1$ | **lone** $pb.password[user_1][url_1]$) **and**
 no $pb.password[user][url]$ **and**
 $pb.password' = pb.password + (user \rightarrow url \rightarrow pwd) \Rightarrow$
 (**all** $user_2, url_2$ | **lone** $pb.password'[user_2][url_2]$)

Proof: An Example

Case 2

Similar.

all pb, user, url, pwd, res |
 Inv[pb] **and** addExceptional[pb, user, url, pwd, res]) $\Rightarrow$
after Inv[pb]

Proof

This kind of reasoning can be somewhat automated

Proof Assistants (Theorem Provers), e.g.:
Isabelle, Coq, HOL4 etc.

Automated Provers: e.g. SAT, SMT solvers etc.
(e.g. MiniSAT, Z3)

Can also be applied to code. (See later in the course.)

Model Checking



Automatically check that every state of a (finite sized) system satisfies some property.

Give a **counterexample** when the check doesn't hold: a state for which the property is false.

Successes:

Hardware Verification

Security Protocol Verification